Team Members: Caleb Bucci, cjb5988, cbucci@utexas.edu

Title: Pass the Aux

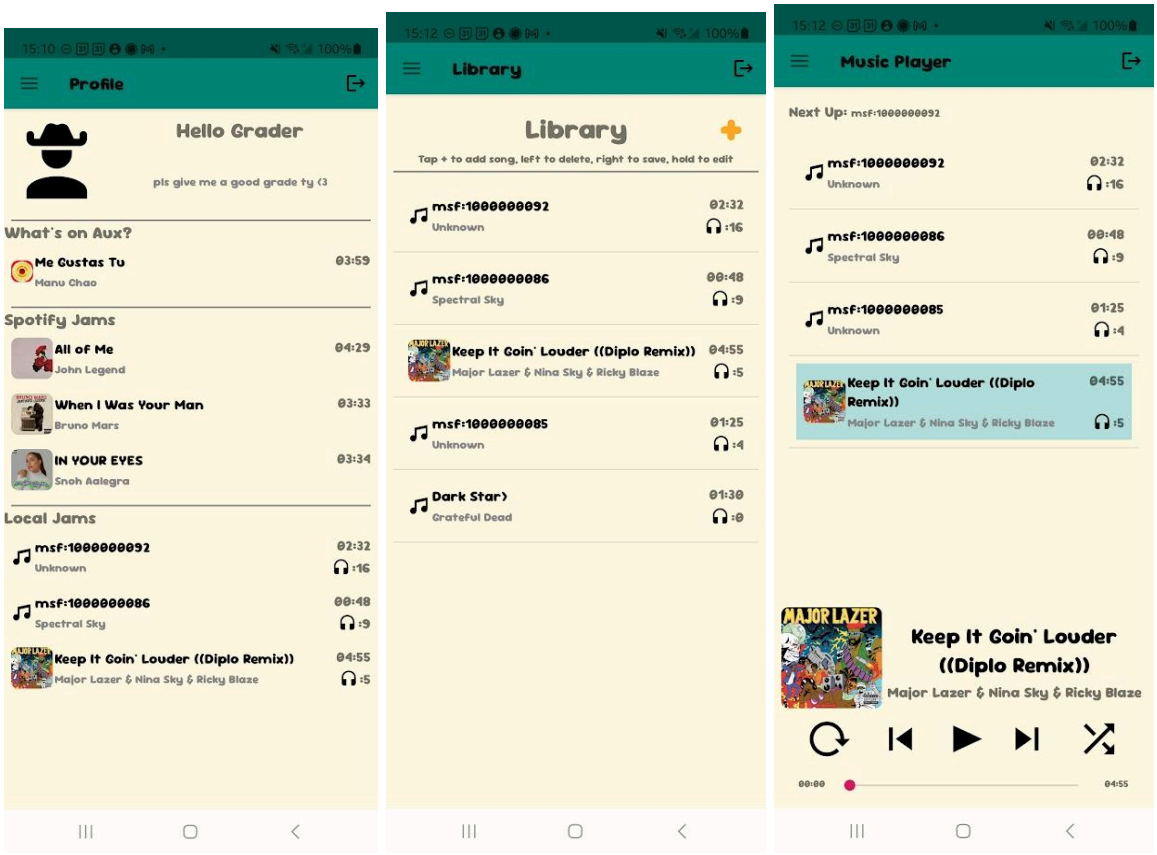
Description: I created a social music sharing and media player application. It connects to the Spotify API and shows your top songs off on your profile that can be shared among users, along with your top songs that you can upload and play locally on the media player.

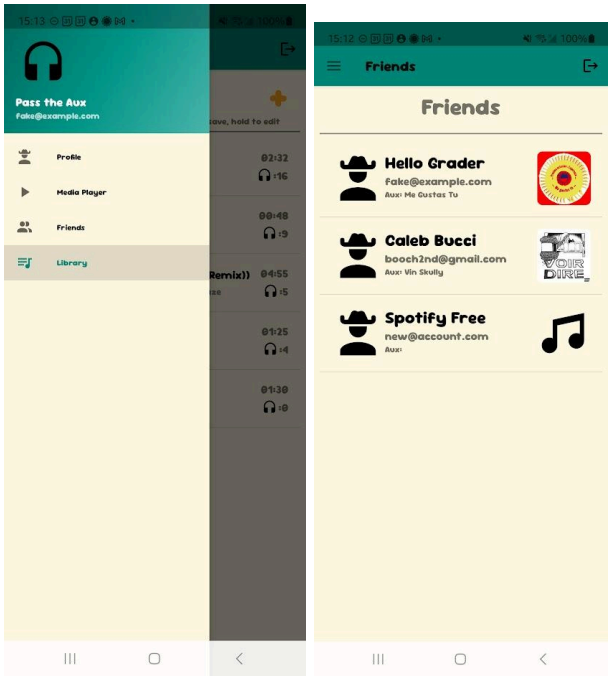
REPO: https://github.com/utap-s25/PassTheAux

VID:

https://drive.google.com/file/d/19IAGfZ1j-LO1ea5cQXDiftuk_k0LwylW/view?usp=sharin

Screenshots of most of the app's major subsystems:





APIs I used:

I heavily integrated the spotify API and Google Firebase/Firestore throughout the entire application. I integrated a navigation drawer to deal with navigating around the app which animates sliding a sidebar over your screen (Can be seen in the screenshots above). I added other gestures such as slide left to delete on songs and slide right to save to library.

Third Party Libraries/Services :

I heavily integrated the AppAuth for Android library (which can be found at https://github.com/openid/AppAuth-Android), the Firebase/Firestore library, and Glide.

I used Glide to load the song covers for songs that had cover art. For the Spotify songs, they contained a coverArtUrl , which I was able to load with Glide easily, so all Spotify songs have artwork displayed. For local MP3 files that the user can upload through the application, I used a MediaMetadataReceiver that was able to retrieve metadata from the specific MP3 file. Occasionally, these had cover art (which can be seen in the media player screenshot above), but if they didn't, then I just used a default album cover. We have used Glide throughout the semester, so I didn't find it challenging, but its ability to load images made that part of the project very seamless.

AppAuth-Android is a library designed to make Oauth 2.0 authentication for android applications more seamless. It is designed to follow the best practices of the Oauth protocols. I used it to authenticate with spotify using the authorization code with PKCE flow(which had its own difficulties that can be seen below in the debugging war story). One of the difficulties I had with this library was setting up the initial authentication and adapting it to fit the spotify API requirements. But after that hurdle, I really appreciated the ease of performing api requests with this library. The spotify API requires the use of access tokens that expire after an hour and then you use a refresh token to get a new token, and the AppAuth library had a helpful performActionWithFreshTokens function that would always use a fresh access token and refresh it when necessary. It made dealing with access tokens much simpler.

I used Google Firebase/Firestore/Storage API fairly extensively. I found authentication to be rather simple since we've done it in the past, and I really appreciated the ability to access files and metadata so easily through firebase. It's what made most of my project possible. I had difficulties creating the data models to properly save everything I wanted to save to the database, and I also had difficulties with setting the rules up properly. I had specific rules about only deleting a song if the current authenticated user had uploaded that specific song, but then I wanted any user to be able to read from it. I also saved my spotify authentication information into firebase so that I could log out and into the application on any device, while keeping my spotify login connected to my firebase authentication. Juggling all of the different rules and database schemas led to some difficulties and many model redesigns (which I could've kept doing if I really wanted to).

List Any Component Generated by AI:

I created my FreshTokenInterceptor primarily using AI. It is a class that I can add to my Retrofit api calls, and it will add a fresh access token to the header of every request that goes through the spotify api. I had asked the AI to help me find a way to add the access token to every api call so that I didn't have to write it into every separate fetch request I made. It came up with the FreshTokenInterceptor, and that was the first I heard of an Interceptor, but it worked perfectly for my use case. After that, I added AppAuth's performActionWithFreshToken function inside the interceptor, and it automatically updated my tokens as needed.

I had AI help me with the idea of using an AlertDialog to edit my bio. I added a onLongClickListener to my bio section on the profile page, and AI created the AlertDialog builder that helps set that up. It's only a few lines but it was perfect for my use case.

I had AI help me with persisting local song information to SharedPreferences by showing me how to use SharedPreferences, how to read from local storage, and how to save to it. It stores my local song metadata and was helpful in implementing my local library for the media player.

Notable UI/UX Features:

The ui was a major focus of my project. Since it was a social music sharing app, I wanted to make it look pretty. I used a color palette to design the application with attractive color combinations, and applied these colors throughout the app through the use of the themes.xml and styles.xml files. I also downloaded a font called Sour_Gummy_Font, which helped make the app look a lot better as well. I used a navigation drawer for the application, which made navigation between fragments seamless, as well as creating a familiar design pattern to users since many real applications use this. The animation of the drawer helped add to the polish of the application. I added swipe features to certain elements of the app that I discussed in the API section of the project above. I redesigned the media player from HW3 to include cover art and artists on each song, as well as redesigning how the current song is shown, which is more in line with modern media players. I added the last played song cover art to each user's profile in the friends section of the app to add a little uniqueness to each profile, on top of updating whenever their latest song updates from Spotify.

Notable Backend Features:

I believe that my most notable backend features include my integration with firebase and creating multi-user shared states. Whenever a user's profile gets updated, it automatically gets sent to firebase where other users can then fetch the updated profile. Each user has their own songs that they listen to in their media players which have local song count listens. The global library has every song uploaded from every user, the ability to download any song uploaded by any user, and then total listen counts for each track for all users. I also believe that the ability to upload any mp3 file from the user's local device that can then be shared to firebase and all other users to download is a very notable feature. It creates a music sharing space similar to soundcloud, and fits in well with the theme of the app which is supposed to be a social music sharing application. I added the ability to listen to the media player no matter where you are in

the application, so you can listen to music and view friends' profiles, your profile, or other songs from the global library.

Most Important Thing I Learned:

It's hard to decide between whether learning how to deal with Oauth authentication for API endpoints or learning how to deal with Firebase was more important, but I believe that learning firebase wins by a close margin. Learning how to heavily integrate a database to store user's information, song information, and user's authentication states, on top of just learning how to design database schemas and set up read/write rules has a huge array of applications. It equipped me with the tools to properly handle mutable shared states, which are very important especially in today's modern applications.

Debugging War Story (Spotify gives me PTSD now):

I'd like to discuss my difficulties with the Spotify API. It had taken me about a week to get the authentication with spotify working. I originally tested connecting to the spotify API through a Python script, which was much easier and I was able to complete that without any outside libraries. However, it was a different story entirely on Android. They had updated their security requirements on Redirect URIs on April 9, 2025 which required them to be https, and mobile applications require the use of custom redirect URIs (eg: passtheaux://callback). I had created this spotify project on their dashboard on the 9th unknowingly 🙃. The application wouldn't redirect me after authentication, instead just saying "Insecure Redirect URI". I ended up finding a bug in their API and had to make a forum post, which is attached below. You can see that others are having the same issue, and it still isn't fixed at the time of upload. I only got it to work because I used an old project I made with the spotify API that had its security requirements around redirect URIs grandfathered in. Without having made this project a few weeks earlier to test the Spotify API, I would not have ever fixed my problem and would've had to change my project entirely. Also, Spotify experienced outages during the time of development, and it made working on the project impossible at times. So I am leaving this project with a bad taste because of Spotify's inconsistency and issues.
https://community.spotify.com/t5/Spotify-for-Developers/INVALID-CLIENT-Insecure-redirect-URI-using-custom-URI/td-p/6919036

How to Run the Project:

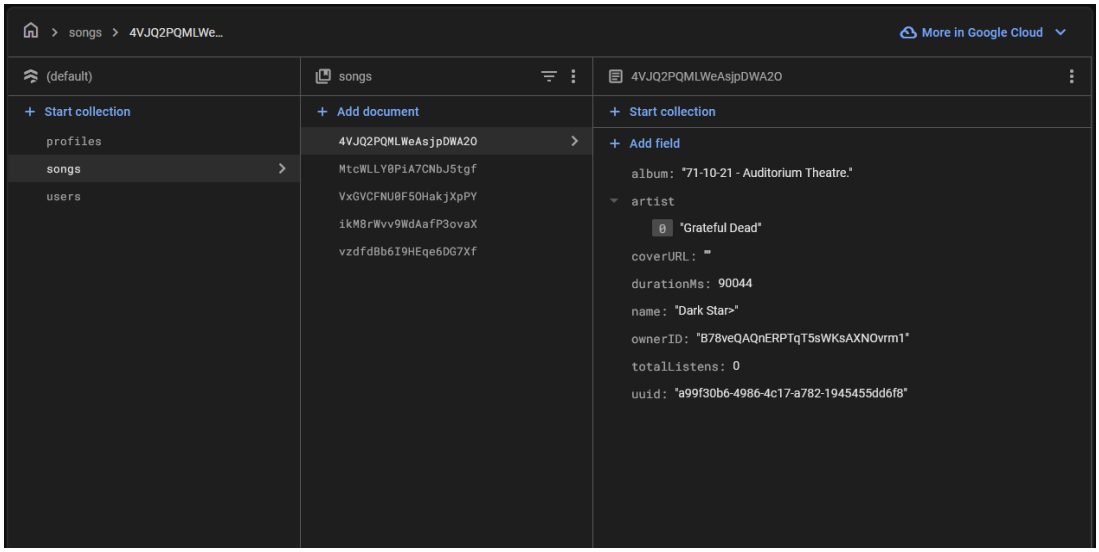
I have a few test users, the password of all of them is 123456. One is booch2nd@gmail.com,

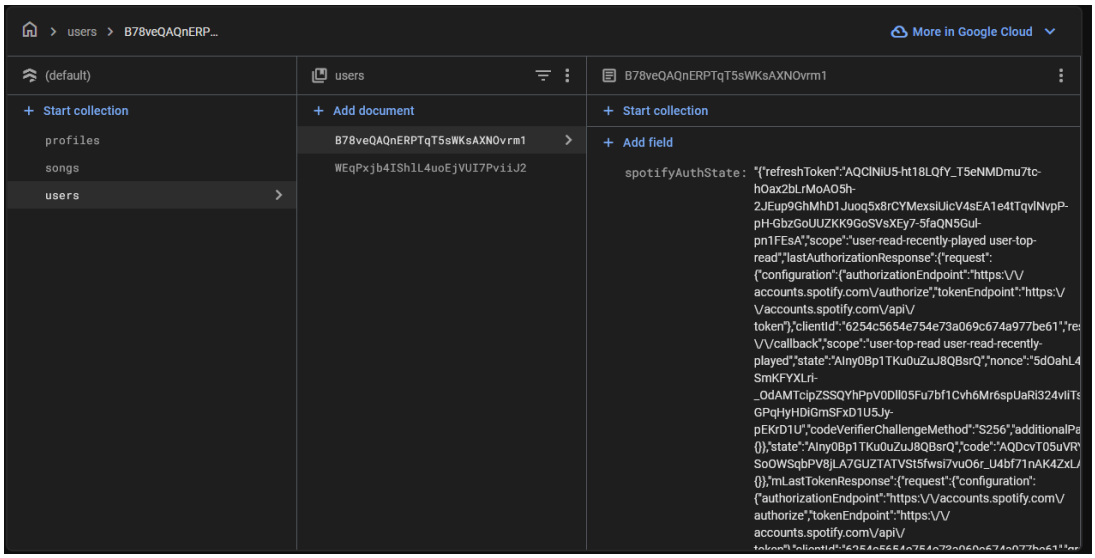
which is connected to my spotify account top songs. Another is fake@example.com, which is connected to a test spotify account I had created for this project. Another is user@name.com, which purposefully has no spotify api account connected to it. Besides that, everything on the project is setup through firebase so you shouldn't have to worry about setting anything else up.

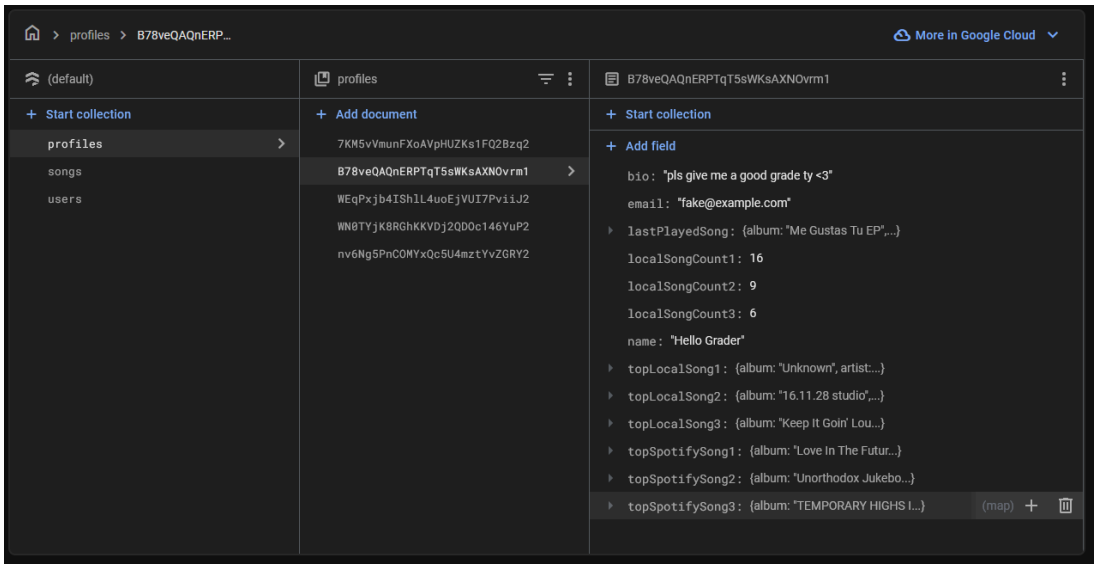
Video Demo Here:

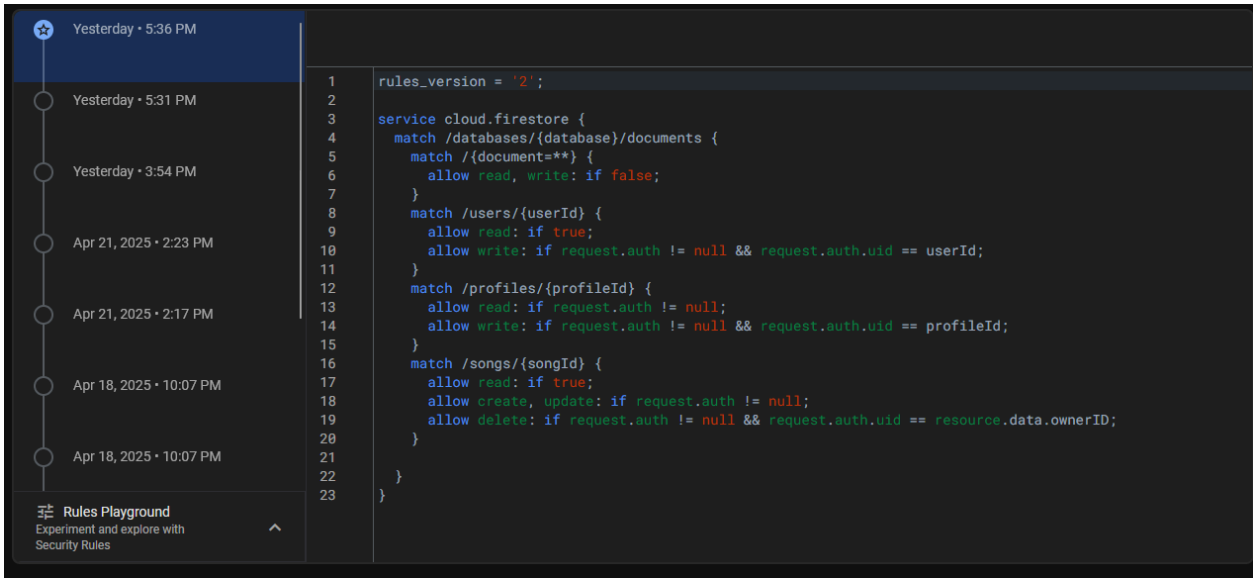
https://drive.google.com/file/d/19IAGfZ1j-LO1ea5cQXDiftuk_k0LwylW/view?usp=sharing

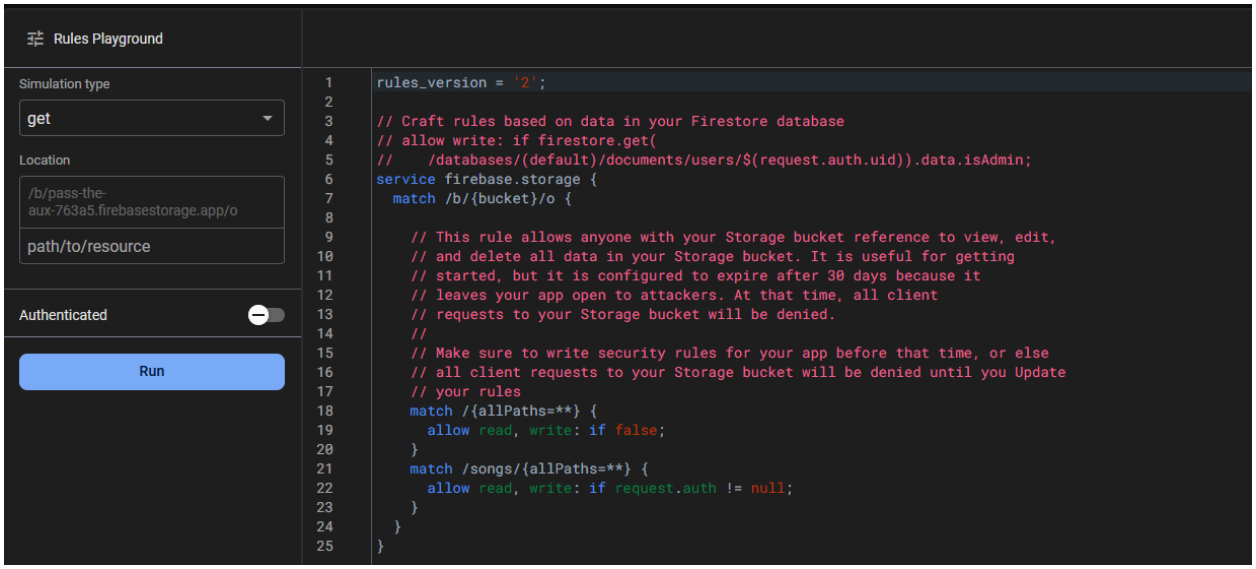
Database Schema:

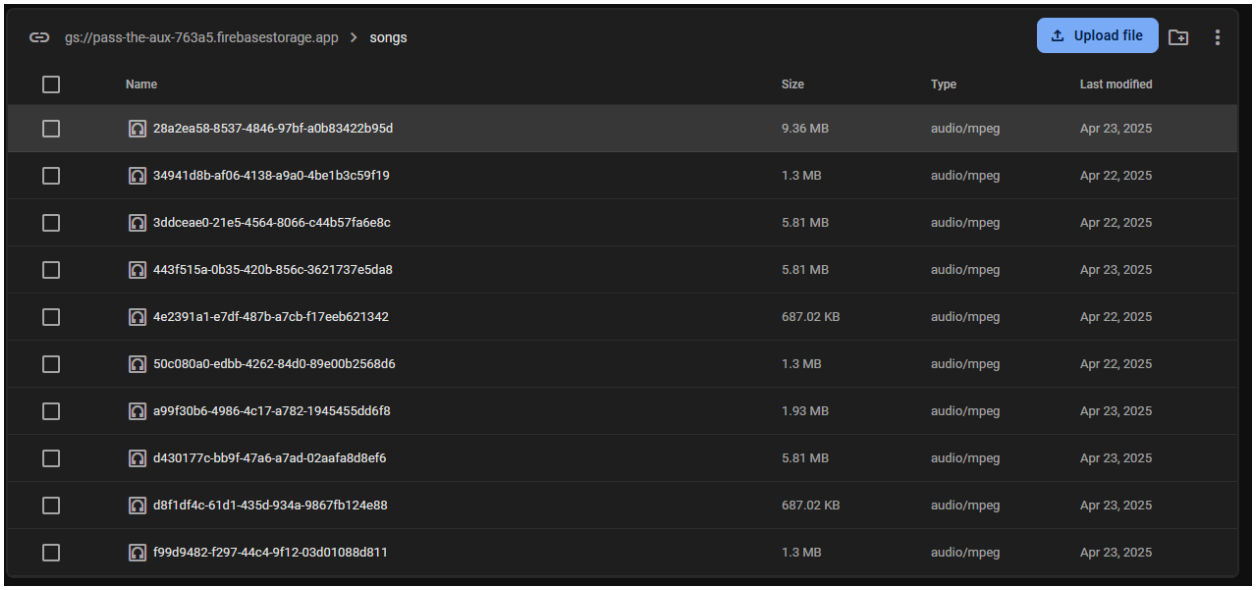

songs > 4VJQ2PQMLWe...
More in Google Cloud
(default)
songs
4VJQ2PQMLWeAsjpDWA2O
Start collection
Add document
Start collection
profiles
songs
users
4VJQ2PQMLWeAsjpDWA2O
MtcWLLY0PiA7CNbJ5tgf
VxGVCFNU0F5OHakjXpPY
ikM8rWvv9WdAafP3ovaX
vzdfdBb6I9HEqe6DG7Xf
Add field
album: "71-10-21 - Auditorium Theatre."
artist
0 "Grateful Dead"
coverURL: ""
durationMs: 90044
name: "Dark Star>"
ownerID: "B78veQAQnERPTqT5sWKsAXNOvrm1"
totalListens: 0
uuid: "a99f30b6-4986-4c17-a782-1945455dd6f8"


users > B78veQAQnERP...
More in Google Cloud
(default)
users
B78veQAQnERPTqT5sWKsAXNOvrm1
Start collection
Add document
Start collection
profiles
songs
users
B78veQAQnERPTqT5sWKsAXNOvrm1
WEqPxjb4IShlL4uoEjVUI7PviiJ2
Add field
spotifyAuthState: "{"refreshToken":"AQClNiU5-ht18LQfY_T5eNMDmu7tc-
hOax2bLrMoAO5h-
2JEup9GhMhD1Juoq5x8rCYMexsiUicV4sEA1e4tTqvlNvpP-
pH-GbzGoUUZKK9GoSVsXEy7-5faQN5Gul-
pn1FEsA","scope":"user-read-recently-played user-top-
read","lastAuthorizationResponse":{"request":
{"configuration":{"authorizationEndpoint":"https:\/\/
accounts.spotify.com\/authorize","tokenEndpoint":"https:\/
\/accounts.spotify.com\/api\/
token"},"clientId":"6254c5654e754e73a069c674a977be61","re
\/\/callback","scope":"user-top-read user-read-recently-
played","state":"AIny0Bp1TKu0uZuJ8QBsrQ","nonce":"5dOahL4
SmKFYXLri-
_OdAMTcipZSSQYhPpV0Dll05Fu7bf1Cvh6Mr6spUaRi324vIiTs
GPqHyHDiGmSFxD1U5Jy-
pEKrD1U","codeVerifierChallengeMethod":"S256","additionalPa
{}},"state":"AIny0Bp1TKu0uZuJ8QBsrQ","code":"AQDcvT05uVR
SoOWSqbPV8jLA7GUZTATVSt5fwsi7vuO6r_U4bf71nAK4ZxL
{}},"mLastTokenResponse":{"request":{"configuration":
{"authorizationEndpoint":"https:\/\/accounts.spotify.com\/
authorize","tokenEndpoint":"https:\/\/
accounts.spotify.com\/api\/


profiles > B78veQAQnERP...
More in Google Cloud
(default)
profiles
B78veQAQnERPTqT5sWKsAXNOvrm1
Start collection
Add document
Start collection
profiles
songs
users
7KM5vVmunFXoAVpHUZKs1FQ2Bzq2
B78veQAQnERPTqT5sWKsAXNOvrm1
WEqPxjb4IShlL4uoEjVUI7PviiJ2
WN0TYjK8RGhKKVDj2QDOc146YuP2
nv6Ng5PnCOMYxQc5U4mztYvZGRY2
Add field
bio: "pls give me a good grade ty <3"
email: "fake@example.com"
lastPlayedSong: {album: "Me Gustas Tu EP",...}
localSongCount1: 16
localSongCount2: 9
localSongCount3: 6
name: "Hello Grader"
topLocalSong1: {album: "Unknown", artist:...}
topLocalSong2: {album: "16.11.28 studio",...}
topLocalSong3: {album: "Keep It Goin' Lou...}
topSpotifySong1: {album: "Love In The Futur...}
topSpotifySong2: {album: "Unorthodox Jukebo...}
topSpotifySong3: {album: "TEMPORARY HIGHS I...}
(map)


Yesterday • 5:36 PM
Yesterday • 5:31 PM
Yesterday • 3:54 PM
Apr 21, 2025 • 2:23 PM
Apr 21, 2025 • 2:17 PM
Apr 18, 2025 • 10:07 PM
Apr 18, 2025 • 10:07 PM
Rules Playground
Experiment and explore with Security Rules
rules_version = '2';
service cloud.firestore {
match /databases/{database}/documents {
match /{document=**} {
allow read, write: if false;
}
match /users/{userId} {
allow read: if true;
allow write: if request.auth != null && request.auth.uid == userId;
}
match /profiles/{profileId} {
allow read: if request.auth != null;
allow write: if request.auth != null && request.auth.uid == profileId;
}
match /songs/{songId} {
allow read: if true;
allow create, update: if request.auth != null;
allow delete: if request.auth != null && request.auth.uid == resource.data.ownerID;
}
}
}


Rules Playground
Simulation type
get
Location
/b/pass-the-aux-763a5.firebasestorage.app/o
path/to/resource
Authenticated
Run
rules_version = '2';
// Craft rules based on data in your Firestore database
// allow write: if firestore.get(
// /databases/(default)/documents/users/$(request.auth.uid)).data.isAdmin;
service firebase.storage {
match /b/{bucket}/o {
// This rule allows anyone with your Storage bucket reference to view, edit,
// and delete all data in your Storage bucket. It is useful for getting
// started, but it is configured to expire after 30 days because it
// leaves your app open to attackers. At that time, all client
// requests to your Storage bucket will be denied.
//
// Make sure to write security rules for your app before that time, or else
// all client requests to your Storage bucket will be denied until you Update
// your rules
match /{allPaths=**} {
allow read, write: if false;
}
match /songs/{allPaths=**} {
allow read, write: if request.auth != null;
}
}
}


gs://pass-the-aux-763a5.firebasestorage.app
songs
Upload file
Name
Size
Type
Last modified
28a2ea58-8537-4846-97bf-a0b83422b95d
9.36 MB
audio/mpeg
Apr 23, 2025
34941d8b-af06-4138-a9a0-4be1b3c59f19
1.3 MB
audio/mpeg
Apr 22, 2025
3ddceae0-21e5-4564-8066-c44b57fa6e8c
5.81 MB
audio/mpeg
Apr 22, 2025
443f515a-0b35-420b-856c-3621737e5da8
5.81 MB
audio/mpeg
Apr 23, 2025
4e2391a1-e7df-487b-a7cb-f17eeb621342
687.02 KB
audio/mpeg
Apr 22, 2025
50c080a0-edbb-4262-84d0-89e00b2568d6
1.3 MB
audio/mpeg
Apr 22, 2025
a99f30b6-4986-4c17-a782-1945455dd6f8
1.93 MB
audio/mpeg
Apr 23, 2025
d430177c-bb9f-47a6-a7ad-02aafa8d8ef6
5.81 MB
audio/mpeg
Apr 23, 2025
d8f1df4c-61d1-435d-934a-9867fb124e88
687.02 KB
audio/mpeg
Apr 23, 2025
f99d9482-f297-44c4-9f12-03d01088d811
1.3 MB
audio/mpeg
Apr 23, 2025

Code Frequency: